Reflexión grupal — turnos-odontologia

Materia: Metodología I — Tecnicatura Universitaria en Programación (UTN FRM)

Trabajo: Trabajo de Integración: del Discovery al primer Change, con Spec-Driven Development y Active Stack

Integrantes: Mariano Chirino, Facundo Quiroga, Andrés Fabre

Reflexión — Mariano Chirino (Discovery)

1. ¿Qué información del Discovery resultó incorrecta, no verificable o inventada por el agente? ¿Cómo lo detecté?

Para el Discovery usé la skill `discovery-research` y el agente generó el informe. Después pasé el archivo de discovery.md a un agente para comprobar que la investigación que hizo el agente fuera correcta, pero encontré estas falencias:

- **DentalSoft:** el informe decía que había planes con precios, pero el sitio no publica precios, solo funcionalidades y beneficios. Lo corregí a "Precio no publicado".
- **Doctoralia:** el informe ponía la lista de espera en dos planes y en la página figuraba solo en el VIP.
- **AgendaPro (centro de ayuda):** no figuraban el IVA ni el costo de WhatsApp, y quedaron como "No evidenciado".

Los tres casos de arriba los comprobé yo abriendo las páginas.

2. ¿En qué etapa usé Ajustar, qué estaba mal y qué habría pasado con Continuar?

Usé Ajustar en el Propose de C-02. Encontré tres problemas:

- Faltaba el requirement de Profesional con matrícula única.
- La `DATABASE_URL` era inconsistente: el diseño decía `postgresql+psycopg://` pero `docker-compose.yml` y `.env.example` usaban `postgresql://`.
- El CI podía saltear los tests de base de datos.

Si se saltearan los tests de GitHub, saldría todo en verde y no sería perceptible si hay algún error. Con la `DATABASE_URL` mal, el proyecto no se habría podido conectar a la base.

3. ¿Qué diferencia noté entre implementar con una especificación aprobada y pedir código con un prompt suelto?

Si apruebo una especificación es porque la leí y razoné que era la correcta para el proyecto. Pedir código con un prompt suelto es una lotería que puede salir mal.

4. ¿Qué parte de la fundación aportó más valor al ciclo OPSX y cuál menos?

Facundo hizo la fundación. Creo que el roadmap aportó más porque tengo una pauta de hacia dónde va el proyecto. No tengo claro cuál aportó menos.

5. Qué hizo cada integrante y en qué etapa

Andrés se encargó de generar la estructura del repositorio, además de hacer un change. Yo, Mariano, me ocupé de hacer el discovery y de corregir algunos otros errores que no tenían que ver con lo que me tocaba. Facundo se ocupó de la fundación: base de conocimiento, roadmap y reglas del agente.

Reflexión — Facundo Quiroga (Fundación)

2. ¿En qué etapa usaron Ajustar, qué estaba mal y qué habría pasado si hubieran apretado Continuar?

Usamos el mecanismo de ajuste/corrección en la Etapa de Fundación. El agente había asumido un stack basado en Node.js, Express, TypeScript y Jest. El error fue no haberle dado al agente la restricción del stack impuesto por la cátedra (Python, FastAPI, SQLAlchemy, PostgreSQL, etc). De haber apretado continuar el agente habría continuado implementando los cambios de código y tests bajo el ecosistema de JavaScript/Node.js. Esto nos habría llevado a entregar el proyecto con un stack equivocado para la correción del trabajo, obligándonos a hacer todo desde cero.

3. ¿Qué diferencia notaron entre implementar con una especificación aprobada y pedir código con un prompt suelto?

La diferencia principal es el control y la cobertura determinista de pruebas. Al usar una especificación aprobada el agente se limitó estrictamente a los requerimientos, como la regla de no solapamiento por profesional/sillón, comportamiento en límites de horario, etc. Esto permitió que la generación de tests pasara sin alucinaciones ni irregularidades. Con un prompt suelto, el agente asume reglas de negocio no pedidas, ignora casos de borde o generar código difícil de validar de forma automatizada.

4. ¿Qué parte de la fundación (knowledge-base, roadmap, reglas) le aportó más valor al ciclo OPSX y cuál menos?

Las reglas del agente "AGENTS.md" y "CLAUDE.md" y el Roadmap "CHANGES.md" nos aportaron mas valor. Las reglas funcionaron como un limite que evitó que el agente rompiera la estructura, inventara dependencias o usara datos reales. El roadmap

permitió acotar el primer cambio a lo esencial sin arrastrar la complejidad del Frontend o Redis antes de tiempo. Por otro lado las partes de menos valor fueron las partes extensas de la base de conocimiento que eran mayormente descriptivas.

5. En grupo: qué hizo cada integrante y en qué etapa.

Configuré la Base de Conocimiento, el Roadmap de cambios, las Reglas del Agente, el registro de skills y formé parte de la reestructuración de la fundación para adaptarla al stack correcto.

Reflexión — Andrés Fabre (Preparación y C-01)

Lo que hicimos en esta sesión

Arrancamos con el proyecto sin fundación (solo `README.md` y `.git`) y terminamos con el **primer change del camino crítico archivado**: `C-01 foundation-setup` con código, tests en verde y spec sincronizada. En el medio pasamos por todo el flujo OPSX: `openspec init` → cierre de fundación → corrección del roadmap → explore → propose → apply → verify → archive.

Lo que más me llevo

1. El estado real lo manda el disco, no el checkbox. `CHANGES.md` marcaba C-01 y C-03 como archivados, pero no había `archive/`, ni `src/`, ni un solo commit de código. Esos `[x]` eran aspiracionales y además imposibles (C-03 depende de C-02, que seguía pendiente). Corregirlos a pendiente antes de codear fue la decisión más importante del día: empezar a construir sobre un plan mentiroso nos habría costado mucho más después.

2. Traer archivos con git no es lo mismo que completar una fase. Hice merge de `main` a mi rama y creí estar al día, pero mi `main` local estaba desactualizado: el archivo nuevo (`.atl/skill-registry.md`) nació después, en un commit que mi merge no podía traer. Y aunque lo hubiera traído, el state del orquestador solo lo escriben sus fases — los archivos huérfanos no cierran nada. Dos lecciones en una: `fetch` antes de asumir que estoy al día, y verificar el estado en vez de suponerlo.

3. En Apply no se improvisa. La regla que me impuse —si no está en el spec, no se inventa en Apply, se vuelve a Propose— funcionó. El ejecutor encontró 5 gaps y los reportó en vez de resolverlos por su cuenta (sin Docker local, una task ya stale, el flag que el CLI no acepta, el 404 que habíamos descartado). Ninguno era grave justamente porque salió a la luz en vez de esconderse en código improvisado.

4. No aprobar a la primera. Revisar el propose contra mis propias pautas antes del apply detectó desvíos reales (escenarios sin GIVEN explícito, sin caso de error). Aprobé

con observación esta vez por ser scaffolding, pero el hábito de leer el spec completo antes de dar luz verde queda.

5. Verify antes de archive, con evidencia. Correr la suite a mano y trazar cada escenario a su test (matriz en `docs/verify/`) me dio una confianza que el "todo verde" de palabra no da. No se archiva nada en rojo.

Lo que haría distinto la próxima vez

- Coordinar con Facundo y Mariano **ramas cortas**: PR → merge a `main` → borrar la rama. Las 4 ramas eternas nos generaron el enredo del merge.
- Definir desde el día uno qué significa cada `[x]` en `CHANGES.md` (código + tests + archive, no intención).
- Instalar las 4 skills de testing que `AGENTS.md` promete y crear el `docker-compose.yml` antes de C-03, que es CRÍTICO y no admite red-green manual improvisado.

Cierre

De cero a fundación cerrada (`step=done`) y C-01 archivado de verdad — esta vez con el `[x]` respaldado por código, 3 tests verdes y una spec en `openspec/specs/foundation/`. Próximo: `C-02 catalogo-recursos`.

Declaración de uso de IA

El trabajo se realizó con agentes de IA (OpenCode con Active Stack) y asistentes de chat (Gemini y otros), que se usaron para generar el informe de Discovery, la base de conocimiento, el roadmap, las especificaciones, el código y los tests de los changes. Todo lo generado fue revisado por el equipo y corregido con Ajustar cuando fue necesario.

- **Mariano:** usé un agente para generar el informe de Discovery y otra IA para revisarlo. También usé un asistente de IA de chat para ordenar mis ideas, revisar que mi reflexión respondiera las preguntas del enunciado y corregir su ortografía. El contenido es mío.
- **Facundo:** se apoyó en Gemini.
- **Andrés:** se apoyó en Gemini.